

Lab 02

ra: 175886

Joao Vitor Saes Casagrande

1. Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada ($ARRIVAL_DELAY > 10$)? Resposta: As duas únicas estatísticas suficientes para a determinação em ($ARRIVAL_DELAY > 10$), são o total de vôos atrasados e o total de vôos válidos.
2. Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`).
 - a. Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US
 - b. Remova observações que tenham valores faltantes em campos de interesse;
 - c. Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
 - d. Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
 - e. A função deve receber apenas dois argumentos.
 - f. `input`: o conjunto de dados (referente ao lote em questão);
 - ii. `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
library(readr)
library(dplyr)
```

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

`filter`, `lag`

Os seguintes objetos são mascarados por 'package:base':

intersect, setdiff, setequal, union

```
getStats <- function(input, pos) {  
  input %>%  
  # a.  
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) %>%  
  # b.  
    filter(!is.na(ARRIVAL_DELAY)) %>%  
  # c.  
    group_by(YEAR, MONTH, DAY, AIRLINE) %>%  
  # d.  
    summarise(  
      n_atrasos = sum(ARRIVAL_DELAY > 10),  
      n_total = n(),  
      .groups = "drop"  
    )  
}  
amostra <- readr::read_csv("flights.csv.zip", n_max = 1000)
```

Rows: 1000 Columns: 31

```
-- Column specification -----  
Delimiter: ","  
chr (11): AIRLINE, TAIL_NUMBER, ORIGIN_AIRPORT, DESTINATION_AIRPORT, SCHEDUL...  
dbl (20): YEAR, MONTH, DAY, DAY_OF_WEEK, FLIGHT_NUMBER, DEPARTURE_DELAY, TAX...  
  
i Use `spec()` to retrieve the full column specification for this data.  
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
```

```
getStats(amostra, pos = 1)
```

```
# A tibble: 4 x 6  
  YEAR MONTH DAY AIRLINE n_atrasos n_total  
  <dbl> <dbl> <dbl> <chr>      <int>   <int>  
1  2015     1     1 AA         23     113  
2  2015     1     1 DL          9     117  
3  2015     1     1 UA         21     103  
4  2015     1     1 US          8      59
```

3. Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.
 - a. Configure o tamanho do lote (chunk) para 100 mil registros;
 - b. Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;
 - c. Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa);

```
colunas <- cols_only(  
  YEAR = col_integer(),  
  MONTH = col_integer(),  
  DAY = col_integer(),  
  AIRLINE = col_character(),  
  ARRIVAL_DELAY = col_double()  
)  
df_estatisticas_parciais <- read_csv_chunked(  
  file = "flights.csv.zip",  
  callback = DataFrameCallback$new(getStats),  
  chunk_size = 100000,  
  col_types = colunas  
)
```

4. Crie uma função chamada `computeStats` que:
 - a. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
 - b. Retorne as informações em um tibble contendo apenas as seguintes colunas:
 - ii. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
 - iii. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo $[0,1]$

```
computeStats <- function(dados_parciais) {  
  dados_parciais %>%  
    # a.  
    group_by(YEAR, MONTH, DAY, AIRLINE) %>%  
    summarise(  
      total_atrasos = sum(n_atrasos),  
      total_voos = sum(n_total),  
      .groups = "drop"  
    ) %>%  
    # b.
```

```

mutate(
  Cia = AIRLINE,
  Data = as.Date(paste(YEAR, MONTH, DAY, sep = "-")),
  Perc = total_atrasos / total_voos
) %>%
select(Cia, Data, Perc)
}
df_final <- computeStats(df_estatisticas_parciais)

```

5. Produza um mapa de calor em formato de calendário para cada Cia. Aérea.
 - a. Instale e carregue os pacotes ggcal e ggplot2.
 - b. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
 - c. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (tibble com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
 - d. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
 - ii. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
 - iii. Retornar para o usuário a base do calendário criada acima.
 - iv. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `cAA`, `cDL`, `cUA` e `cUS`.
 - e. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Título")`.

```

#a.
library(ggplot2)
library(ggcal)
#b.
pal <- scale_fill_gradient(low = "#4575b4", high = "#d73027", name = "% Atraso")

#c,
baseCalendario <- function(stats, cia) {
  df_sub <- stats %>% filter(Cia == cia)
  cal <- ggcal(df_sub$Data, df_sub$Perc)
  return(cal)
}

```

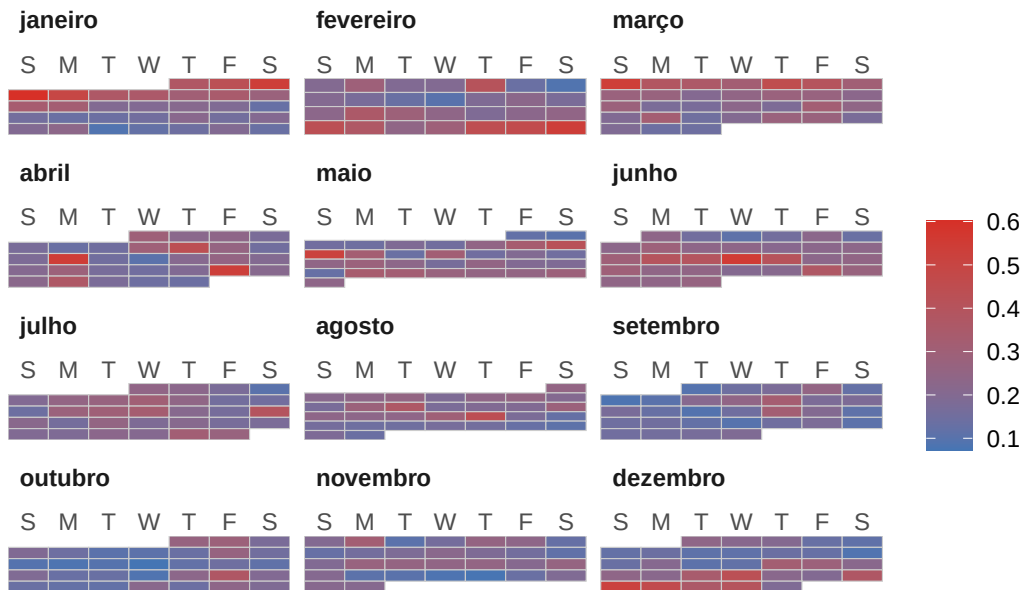
```

}
#d.
cAA <- baseCalendario(df_final, "AA")
cDL <- baseCalendario(df_final, "DL")
cUA <- baseCalendario(df_final, "UA")
cUS <- baseCalendario(df_final, "US")
#e.
plot_AA <- cAA + pal + ggtitle("Percentual de Atrasos (>10 min) - American Airlines (AA)")
plot_DL <- cDL + pal + ggtitle("Percentual de Atrasos (>10 min) - Delta Air Lines (DL)")
plot_UA <- cUA + pal + ggtitle("Percentual de Atrasos (>10 min) - United Airlines (UA)")
plot_US <- cUS + pal + ggtitle("Percentual de Atrasos (>10 min) - US Airways (US)")

plot_AA

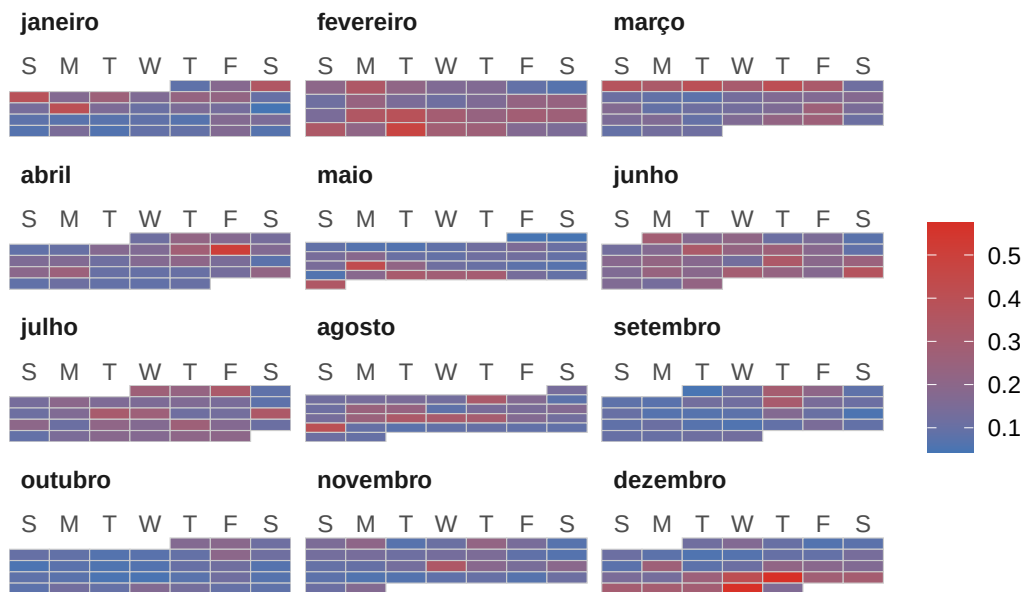
```

Percentual de Atrasos (>10 min) – American Airlines (AA)



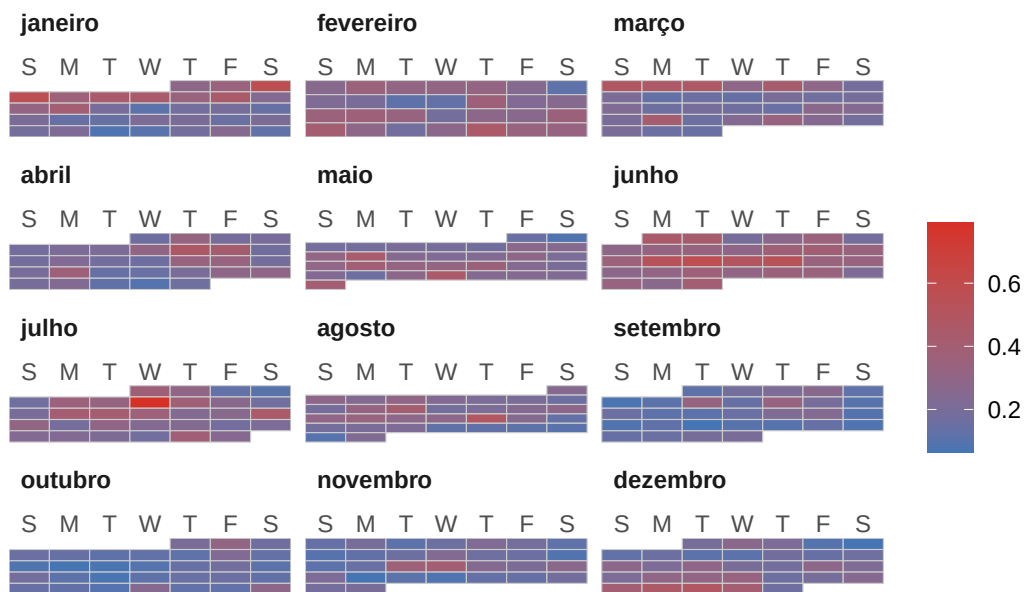
```
plot_DL
```

Percentual de Atrasos (>10 min) – Delta Air Lines (DL)



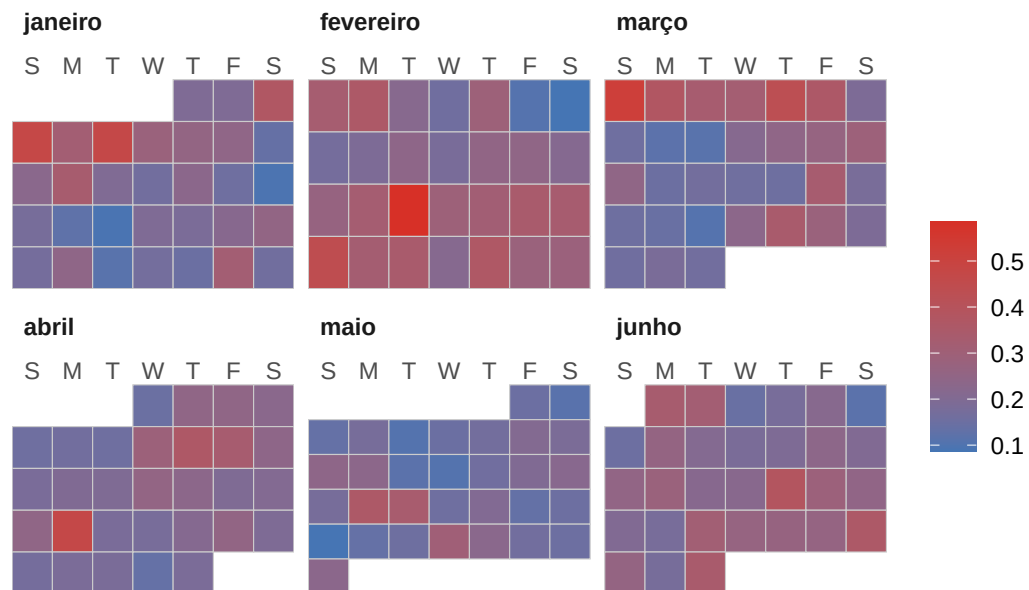
plot-UA

Percentual de Atrasos (>10 min) – United Airlines (UA)



```
plot_US
```

Percentual de Atrasos (>10 min) – US Airways (US)



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-24 14:11:08 -03"
```